

**Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Нижевартовский государственный университет»**

**Лабораторный практикум по робототехнике
Часть 2**

© М.В. Слива

Нижевартовск 2025

Содержание

Лабораторная работа №1. Светодиодная матрица	3
Лабораторная работа №2. Светодиодная лента формата NeoPixel	9
Лабораторная работа №3. Двустрочный (четырёхстрочный) LCD-дисплей	13
Лабораторная работа №4. Двухосный джойстик	17
Лабораторная работа №5. Работа с мини клавиатурой 4x4	20
Лабораторная работа №6. Работа с графическими дисплеями	23
Лабораторная работа №7. ИК-модуль и пульт ДУ	28
Лабораторная работа №8. Передача данных с компьютера на устройство и обратно	33

Лабораторная работа №1. Светодиодная матрица

Представляет собой матрицу, состоящую из группы светодиодов – с общими анодами в рядах и общими катодами в столбцах (или наоборот).

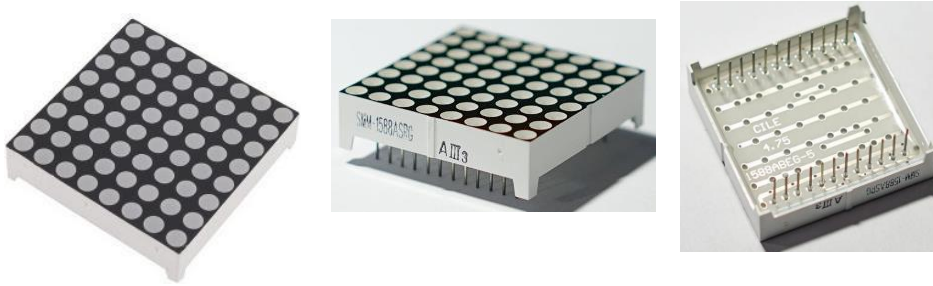


Рис. 1. Светодиодная матрица

Но подключать такую матрицу напрямую к Arduino не очень удобно (рис. 2), требует 16 проводов.

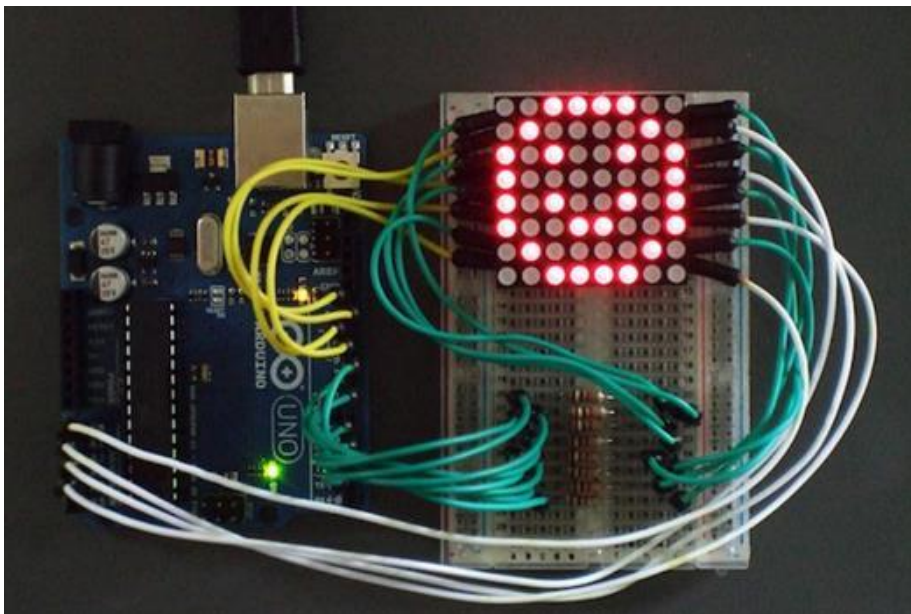


Рис. 2. Подключение светодиодной матрицы к Arduino напрямую

Поэтому чаще всего используют подключение с помощью специальной платы (рис. 3).

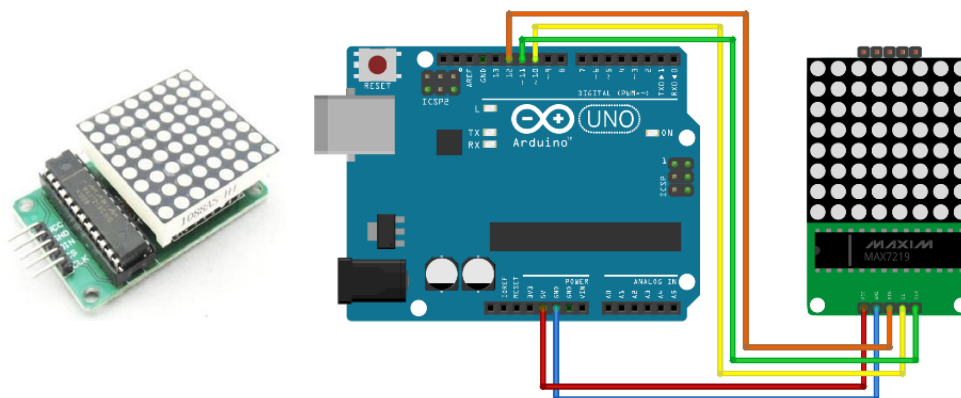


Рис. 3. Подключение светодиодной матрицы с помощью специальной платы

В Wokwi эта схема может выглядеть примерно как на рис. 4.

Если при добавлении матрицы (элемент LED Dot Matrix (MAX7219)) она появится слишком длинной, то на вкладке diagram.json нужно найти ее описание в разделе parts

```
{
  "type": "wokwi-max7219-matrix",
  "id": "matrix1",
  "top": -153,
  "left": -92.19,
  "attrs": { "chain": "4" }
}
```

и в строке "attrs": { "chain": "4" } поменять 4 на 1.

Подключение в данном случае происходит по протоколу SPI:

оранжевый провод (DIN) → пин 12

зеленый провод (CLK) → пин 11

желтый провод (CS) → пин 10

Для написания кода по управлению матрицей нужно подключить библиотеку LedControl.

В Wokwi это делается на вкладке Library Manager или в специальном файле проекта libraries.txt.

Для управления отдельными светодиодами на матрице удобно использовать функцию

```
setLed(n, row, col, true);
```

где n – номер матрицы (т.к. их можно подключать друг к другу конвейером),

row – номер ряда (0 – верхний),

col – номер колонки (0 – крайний правый),

true – включить светодиод (false для выключения).

Например, код для включения некоторых светодиодов:

```
#include "LedControl.h"
LedControl lc = LedControl(12,11,10,1); //создаем объект для управления матрицей

void setup() {
  lc.shutdown(0, false); // включаем матрицу
  lc.setIntensity(0, 8); // устанавливаем яркость(0-15)
  lc.clearDisplay(0); // очищаем матрицу
}
```

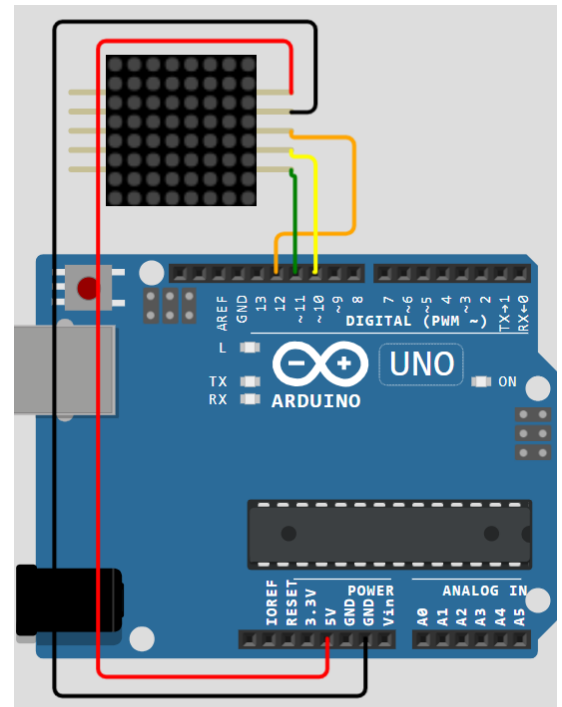
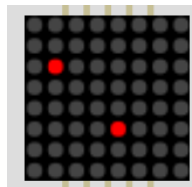


Рис. 4. Подключение светодиодной матрицы к Arduino в Wokwi

```

}
void loop() {
  lc.setLed(0, 2, 6, true);
  lc.setLed(0, 5, 3, true);
}

```



Можно работать со светодиодами в цикле, создавая эффект анимации:

```

for (int row=0; row<8; row++) { //во внешнем цикле идем по строке
  for (int col=7; col>=0; col--) { //во внутреннем по столбцам
    lc.setLed(0, row, col, true); //и включаем по 1 светодиоду
    delay(100); //с задержкой в 100 мс
  }
}

for (int row=0; row<8; row++) { //и потом то же самое для выключения светодиодов
  for (int col=7; col>=0; col--) {
    lc.setLed(0, row, col, false);
    delay(100);
  }
}

```

Можно заполнять сразу целый ряд или столбец.

Для ряда используется функция `setRow(n, i, 255)`, где `n` – номер матрицы (т.к. их можно подключать друг к другу конвейером), `i` – номер ряда (нумеруются сверху вниз), 255 – признак заполнения всего ряда.

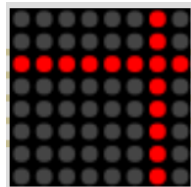
Для столбца используется функция `setColumn(n, i, 255)`, параметры аналогичны предыдущей функции, только столбцы нумеруются справа налево.

Например:

```

lc.setRow(0, 2, 255);
lc.setColumn(0, 1, 255);

```

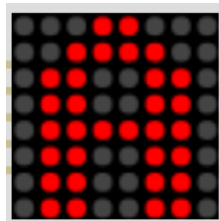


Можно выводить битовые массивы на матрице в качестве пиксельной картинки:

```

byte letterA[8] = { //создаем битовый массив, в котором 1 означает включение
  00011000,        // светодиода на нужном месте матрицы, 0 – выключение,
  00111100,        // в данном случае изображается буква А
  01100110,
  01100110,
  01111110,
  01100110,
  01100110,
  01100110
};

```



```

};
//создаем функцию для вывода строки массива в качестве ряда матрицы
void drawLetter(int row, byte letterData[]) {
  for (int i = 0; i < 8; i++) { //цикл по строкам массива
    lc.setRow(0, row + i, letterData[i]); //выводим ряд матрицы согласно битовой строке
  }
}

```

```

}
}

```

```

void loop() {
  drawLetter(0, letterA); // выводим битовый массив для формирования картинки
}

```

Если использовать реальную плату Arduino, то удобнее использовать матрицы от фирмы Amperka.



Такая матрица подключается 4 проводами по протоколу i2c:

- пин V — питание линии;
- пин G — земля линии;
- пин D — пин передачи данных, подключить к пину SDA микроконтроллера;
- пин C — пин линии тактирования шины i2c, подключить к пину SCL.

Можно подключить через плату расширения Troyka Shield или Troyka Slot Shield (рис. 4.1).

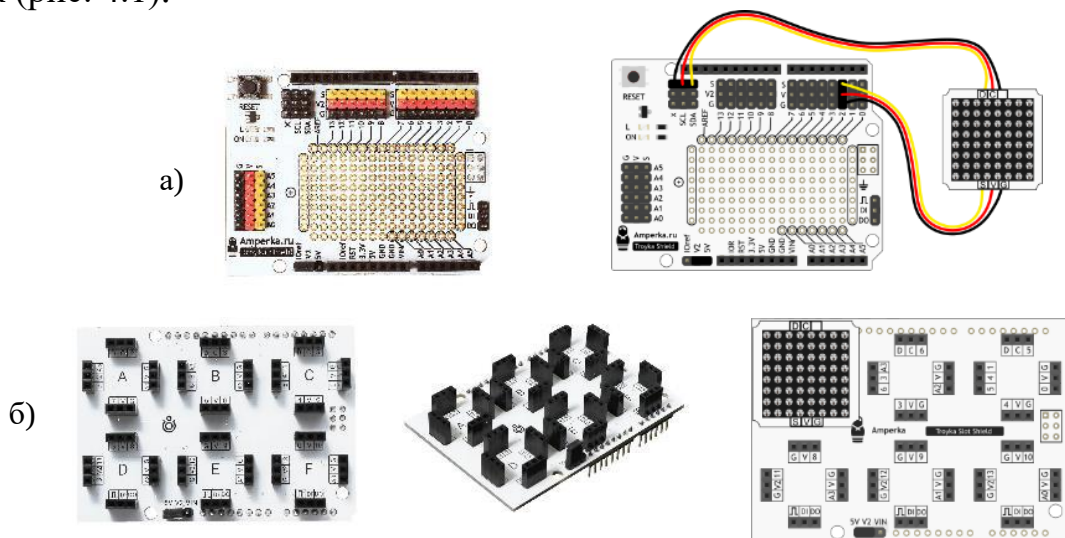


Рис. 4.1. Подключение матрицы с использованием

- a) Troyka Shield;
b) Troyka Slot Shield.

Для написания скетчей понадобится библиотека [TroykaLedMatrix](#).

Пример кода для вывода случайных пикселей (будем поочерёдно добавлять и удалять случайные пиксели):

```

#include "Wire.h" //библиотека для работы I2C

```

```

#include "TroykaLedMatrix.h" //библиотека для работы со светодиодной матрицей

```

```

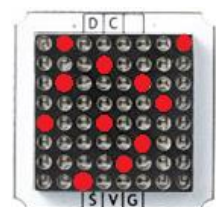
TroykaLedMatrix matrix; // создаём объект matrix для работы с матрицей

```

```

void setup()
{
  matrix.begin(); // начало работы с матрицей
  matrix.clear(); // очищаем матрицу
}

```



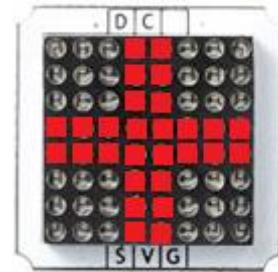
```
void loop()
{
    matrix.drawPixel(random(8),random(8)); // заполняем матрицу случайным пикселем
    matrix.clearPixel(random(8),random(8)); // очищаем на матрице случайный пиксель
    delay(50); // ждём 50 мс
}
```

Пример кода для вывода изображения креста:

```
#include "Wire.h" // библиотека для работы I²C
#include "TroykaLedMatrix.h" // библиотека для работы со светодиодной матрицей
TroykaLedMatrix matrix; // создаём объект matrix для работы с матрицей
const uint8_t cross[] { // массив картинки креста
    0b00011000,
    0b00011000,
    0b00011000,
    0b11111111,
    0b11111111,
    0b11111111,
    0b00011000,
    0b00011000,
    0b00011000,
};

void setup(){
    matrix.begin();
    matrix.clear();
    matrix.drawBitmap(cross); // отображаем на матрице крест
}

void loop() { }
```



Пример вывода буквы:

```
#include "Wire.h"
#include "TroykaLedMatrix.h"
TroykaLedMatrix matrix;

void setup()
{
    matrix.begin();
    matrix.clear();
    matrix.selectFont(FONT_8X8_BASIC); // устанавливаем шрифт
    matrix.drawSymbol('A'); // выводим букву A
}

void loop() { }
```

Задания

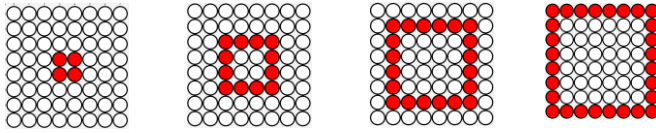
I. Обязательные:

1. Вывести на матрице буквы своего имени или фамилии по одной с небольшой задержкой.
2. Вывести на матрице изображение по своему варианту:
 - 1) смайл
 - 2) грустный смайл
 - 3) домик
 - 4) дерево
 - 5) треугольник
 - 6) квадрат

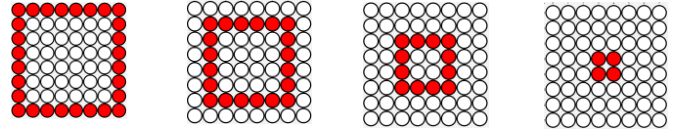
7) ромб

II. Индивидуальные (выбирается одно):

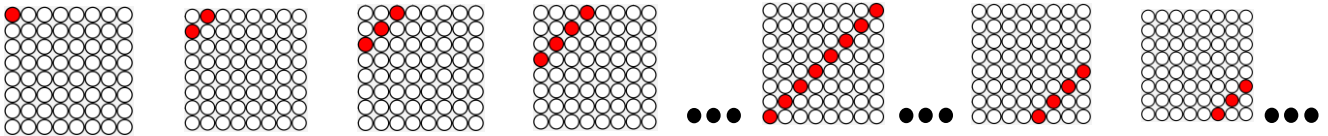
1. Движение точки по рядам (сначала по первому, потом по второму и т.д., потом снова по первому).
2. Движение точки по столбцам (сначала по первому, потом по второму и т.д., потом снова по первому).
3. Движение точки сначала по главной диагонали, потом по побочной.
4. Постепенное увеличение квадрата:



5. Постепенное уменьшение квадрата:



6. Перемещение линии по диагонали:



7. Движение точки по краю матрицы (от верхнего левого к верхнему правому, потом вниз к нижнему правому, потом к нижнему левому, потом к верхнему левому).

Лабораторная работа №2. Светодиодная лента формата NeoPixel

Представляет собой группу RGB-светодиодов (рис. 5). Могут быть разного размера и форм.

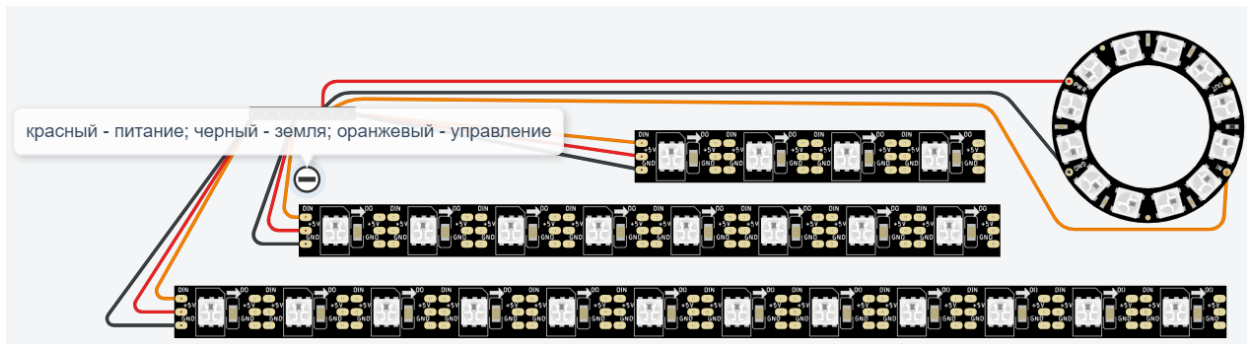


Рис. 5. Светодиодные ленты формата NeoPixel

Основная особенность – для подключения нужно всего 3 провода – земля, питание и управление. Также можно соединять ленты друг с другом для получения нужного формата и размера, например, можно сделать квадратную матрицу из нескольких лент (рис. 5а).

Для работы с такой лентой в Wokwi удобно вставить ее в проект через файл diagram.json (вкладка рядом с вкладкой с кодом). Например, для добавления NeoPixel-матрицы 4x4 понадобится примерно следующий код в секции "parts" (не забудьте поставить запятую после предыдущего кода):

```
{
  "type": "wokwi-neopixel-canvas",
  "id": "neopixels1",
  "top": -160,
  "left": 115,
  "attrs": { "rows": "4", "cols": "4", "matrixBrightness": "5", "pixelate": "1" }
}
```

Если до этого в проекте была только одна плата Arduino, то код в разделе parts будет примерно следующий:

```
"parts": [ { "id": "uno", "type": "wokwi-arduino-uno" },
  {
    "type": "wokwi-neopixel-canvas",
    "id": "neopixels1",
    "top": -160,
    "left": 115,
    "attrs": { "rows": "4", "cols": "4", "matrixBrightness": "5", "pixelate": "1" }
  }
],
```

Параметр "rows" отвечает за количество рядов, параметр "cols" отвечает за

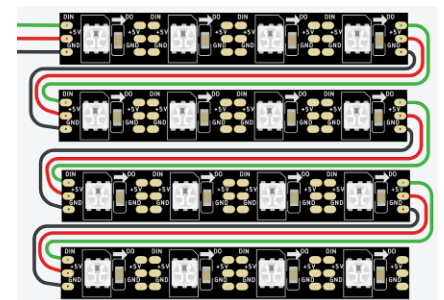


Рис. 5а. Квадратная матрица 4x4, зеленый - управление

количество столбцов.

Сама матрица добавится на рабочее поле в виде черного прямоугольника (рис. 6). И пины подключения будут не видны, их обозначение появляется при наведении курсора мыши в центр нижней стороны прямоугольника (рис. 6).

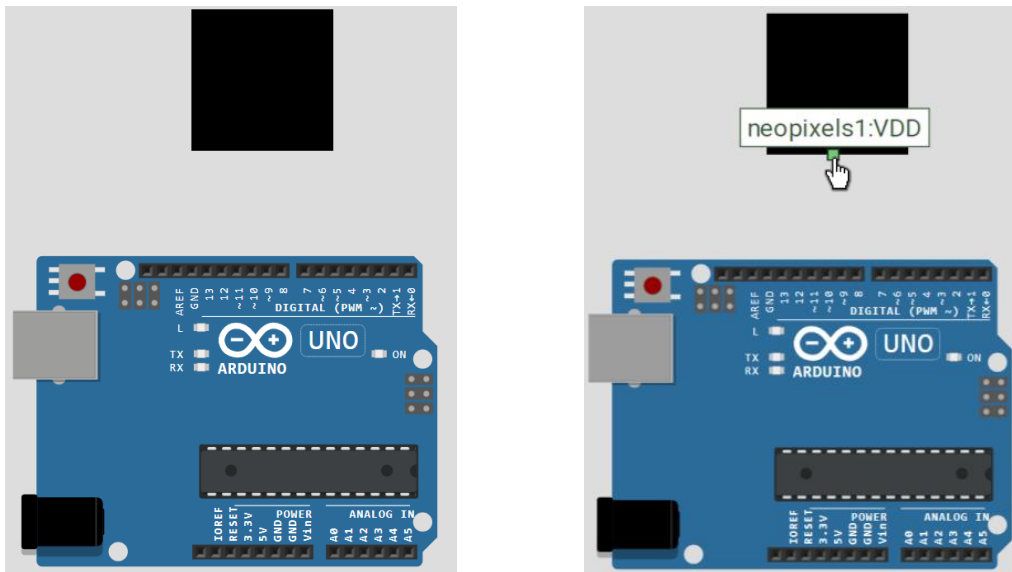


Рис. 6. Появление пинов для подключения матрицы формата NeoPixel при наведении курсора мыши

Назначение пинов следующее (идут слева направо):

DOUT – подключение другой матрицы (если необходимо), VDD – питание (5в), VSS – земля, DIN – управление.

Для написания скетчей понадобится библиотека [Adafruit NeoPixel](https://www.adafruit.com/library/adafruit-neopixel). Подключить ее можно на вкладке Library Manager (кнопка +, в строке поиска написать Adafruit NeoPixel) или на вкладке libraries.txt (в свободной строке написать Adafruit NeoPixel). Основные функции библиотеки:

`Adafruit_NeoPixel(LED_COUNT, MATRIX_PIN, NEO_GRB + NEO_KHZ800)` – конструктор для создания объекта для управления лентой, где `LED_COUNT` – кол-во светодиодов, `MATRIX_PIN` – пин подключения управляющего входа ленты, третий параметр – стандартный (не меняется). Пример использования:

```
Adafruit_NeoPixel matr = Adafruit_NeoPixel(16, 4, NEO_GRB + NEO_KHZ800);
```

Все остальные функции вызываются у созданного объекта:

`begin()` – вызывается один раз в `setup` для начала работы с лентой;

`setBrightness(n)` – устанавливает яркость ленты (от 0 до 255);

`setPixelColor(i, r, g, b)` – устанавливает цвет светодиода с номером `i` в формате RGB;

`show()` – применяет установленные другими функциями параметры (нужно вызывать каждый раз, как что-то меняем – цвет нужного светодиода и т.д.).

Пример кода для постепенного заполнения матрицы из 16 светодиодов (4x4) красным цветом:

```
#include <Adafruit_NeoPixel.h>
#define MATRIX_PIN 4 //управляющий вход подключаем к 4 пину Arduino
```

```

#define LED_COUNT    16    // количество светодиодов в матрице
// создаём объект класса Adafruit_NeoPixel
Adafruit_NeoPixel matr = Adafruit_NeoPixel(LED_COUNT, MATRIX_PIN, NEO_GRB + NEO_KHZ800);

void setup() {
    matr.begin();
    matr.setBrightness(100);
    matr.setPixelColor(0, 255,0,0);
    matr.show();
}

void loop(){
    for (int i = 0; i < LED_COUNT; i++){
        matr.setPixelColor(i, 255,0,0);
        matr.show();
        delay(200);
    }
}

```

Можно немного модифицировать код, добавив в конце цикла после `delay(200);` строки

```

    matr.setPixelColor(i, 0,0,0);
    matr.show();

```

Это сделает эффект перемещающегося красного пикселя.

Пример кода для постепенного заполнения матрицы 4x4 точками разного цвета (сначала все красного, потом все зеленого, потом – синего и снова по кругу), в предыдущем примере меняем метод `loop`:

```

void loop(){
    for (int j = 0; j<3; j++) { //внешний цикл для смены цвета
        int r = (j == 0)? 255 : 0; // через тернарную операцию задаем значения
        int g = (j == 1)? 255 : 0; // переменных, отвечающих за нужный цвет
        int b = (j == 2)? 255 : 0; //
        for (int i = 0; i < LED_COUNT; i++){ //внутренний цикл для прохода по всем светодиодам
            matr.setPixelColor(i, r, g ,b);
            matr.show();
            delay(200);
        }
    }
}

```

Разберем более комплексный пример – вывод точечного изображения креста на матрице 8x8:

```

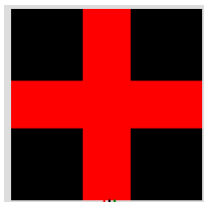
#include <Adafruit_NeoPixel.h>
#define MATRIX_PIN 4
#define LED_COUNT 64
Adafruit_NeoPixel matrix = Adafruit_NeoPixel(LED_COUNT, MATRIX_PIN, NEO_GRB + NEO_KHZ800);

char* cross[8] { //картинка в виде массива строк с 1 и 0
    "00011000",
    "00011000",
    "00011000",
    "11111111",
    "11111111",
    "00011000",
    "00011000",
    "00011000"
};

//создаем функцию, которая принимает в качестве параметров массив с картинкой
//и цвет в формате 32-битового целого без знака, этот параметр будем передавать в метод
//matrix.setPixelColor
void showPic(char* pic[8], uint32_t c) {
    for (uint16_t i = 0; i < matrix.numPixels(); i++) { //цикл по всем точкам матрицы
        //если в нужном ряду на нужном месте стоит 1, то выводим на матрице точку нужного цвета
        if (pic[i/8][i%8]=='1') matrix.setPixelColor(i, c);
        else matrix.setPixelColor(i, matrix.Color(0,0,0)); //иначе пустая точка
    }
}

void setup() {

```



```

matrix.begin();
matrix.setBrightness(255); //максимальная яркость
showPic(cross, matrix.Color(255,0,0)); //рисует крест красным цветом
matrix.show();
}

void loop() { }

```

В этом примере используется функция `matrix.Color(255,0,0)`, которая преобразует цвет в формате RGB в целую переменную типа `uint32_t`, это 32-битовый целый без знака, в таком виде удобно передавать цвет в созданную функцию `showPic` и в ней передавать этот цвет в метод `setPixelColor`.

Задания (с использованием NeoPixel матрицы 8x8)

I. Обязательная:

Сделать задачу из [лаб.раб.1 часть 2](#), с изменением цвета при каждом движении.

II. Индивидуальная (выбирается одно):

1. Схема состоит из матрицы NeoPixel, кнопки и потенциометра. В начальном состоянии в матрице горит красным верхний левый сегмент. Потенциометр меняет цвет элемента, кнопка перемещает элемент ниже по основной диагонали, когда она заканчивается – возвращаемся на первый элемент диагонали.
2. Схема состоит из матрицы NeoPixel и 2-х кнопок. В начальном состоянии в матрице горит красным верхний левый сегмент. Первая кнопка перемещает зажженный сегмент по строке, вторая – по столбцу. При каждом перемещении меняется цвет сегмента.
3. Схема состоит из матрицы NeoPixel и 2-х кнопок. В начальном состоянии в матрице горит красным верхний левый сегмент. Первая кнопка перемещает зажженный сегмент по строке, вторая – меняет цвет (красный, зеленый синий по кругу в зависимости от кол-ва нажатий). При перемещении на последний элемент строки следующим загорается первый сегмент следующей строки.
4. Схема состоит из матрицы NeoPixel, кнопки и потенциометра. В начальном состоянии в матрице горит красным верхний левый сегмент. Потенциометр меняет цвет элемента, кнопка перемещает элемент на строку ниже, когда строки заканчиваются – переходим вверх на первый элемент следующего столбца.
5. Схема состоит из матрицы NeoPixel, 2-х потенциометров и кнопки. В начальном состоянии в матрице горит красным верхний левый сегмент. Первый потенциометр перемещает зажженный сегмент по строке, второй – по столбцу. Кнопка меняет цвет сегмента.
6. Схема состоит из матрицы NeoPixel, кнопки и потенциометра. В начальном состоянии в матрице горит красным верхний правый сегмент. Потенциометр меняет цвет элемента, кнопка перемещает элемент ниже по побочной диагонали, когда она заканчивается – возвращаемся на первый элемент побочной диагонали (верхний правый сегмент).
7. Схема состоит из матрицы NeoPixel, 2-х кнопок и потенциометра. В

начальном состоянии в матрице горит красным верхний левый сегмент. Потенциометр меняет цвет элемента, первая кнопка перемещает элемент вправо, когда верхняя строка заканчивается – идем вниз по последнему столбцу, когда он закончится – идем по последней строке влево, потом – по первому столбцу вверх. Вторая кнопка перемещает сегмент в обратном направлении.

Лабораторная работа №3. Двустрочный (четырёхстрочный) LCD-дисплей

Этот монохромный дисплей имеет опциональную подсветку и может отображать 2 или 4 строки по 16 или 20 символов (рис. 7). Разрешение символов — 5x8 точек.

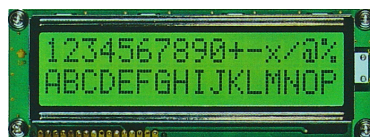


Рис. 7. Двустрочный LCD-дисплей.

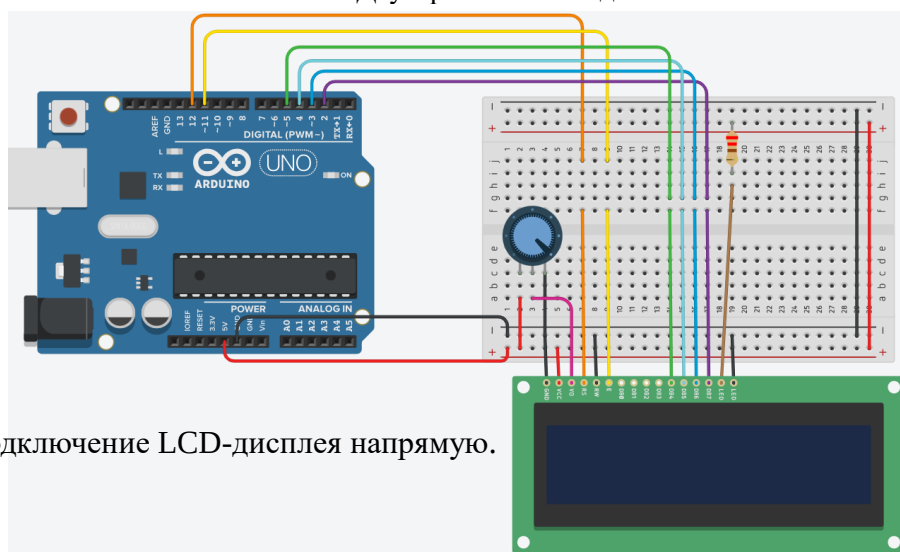


Рис. 8. Подключение LCD-дисплея напрямую.

Экран имеет 16 контактов для подведения питания и взаимодействия с управляющей электроникой. Для управления выводом дисплея возможно использовать только 6 из них (рис. 8).

- 1 — **Vss**, земля $\Rightarrow$ GND
- 2 — **Vdd**, питание $\Rightarrow$ +5 V
- 3 — **Vo**, управление контрастностью напряжением $\Rightarrow$ выход потенциометра
- 4 — **RS**, выбор регистра $\Rightarrow$ пин 12 Arduino
- 5 — **R/W**, чтение/запись $\Rightarrow$ земля (режим записи)
- 6 — **E**, он же Enable, строб по спаду $\Rightarrow$ пин 11 Arduino
- 7-10 — **DB0-DB3**, младшие биты 8-битного интерфейса; не подключены

11-14 — **DB4-DB7**, старшие биты интерфейса ⇒ пины 5-2 Arduino

15 — **A**, питание для подсветки ⇒ +5 В

16 — **K**, земля для подсветки ⇒ GND

Пример кода:

```
#include <LiquidCrystal.h> /* Подключаем библиотеку для работы с LCD */
/* Создаём объект LCD-дисплея, используя конструктор класса LiquidCrystal
 * с 6-ю аргументами. Указываем, к каким пинам Arduino подключены выводы дисплея:
 * RS, E, DB4, DB5, DB6, DB7*/
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup()
{
  lcd.begin(16, 2); /* Инициализируем дисплей: 2 строки по 16 символов */
  lcd.print("hello, world!"); /* Выводим на дисплей традиционную фразу (: */
}

void loop()
{
  // Устанавливаем курсор в 1-й столбец 2-й строки.
  // Нумерация идёт с нуля,
  lcd.setCursor(0, 1); // первым аргументом идёт
  // номер столбца.
  /* Выводим на дисплей число секунд, прошедших с
  момента старта Arduino */
  lcd.print(millis() / 1000);
}
```

В виртуальной среде Wokwi есть возможность более простого варианта подключения такого дисплея – через i2c (рис. 9).

У Arduino есть по 2 выхода i2c с каждой стороны: 2 крайние слева сверху и 2 крайние справа снизу, если Arduino расположена как на рис.9.

Подключать можно в любой из них, соблюдая порядок: SCL – в крайний, SDA – в следующий.

Код для проверки работы дисплея может быть примерно следующий:

```
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27,16,2); // Задаем адрес и размерность дисплея

void setup()
{
  lcd.init(); // Инициализация lcd
  lcd.backlight(); // Включаем подсветку
  lcd.setCursor(3,0); // перемещаем курсор на 3 позицию в первой строке
  lcd.print("Hello, world!"); // Выводим текст
}

void loop()
```

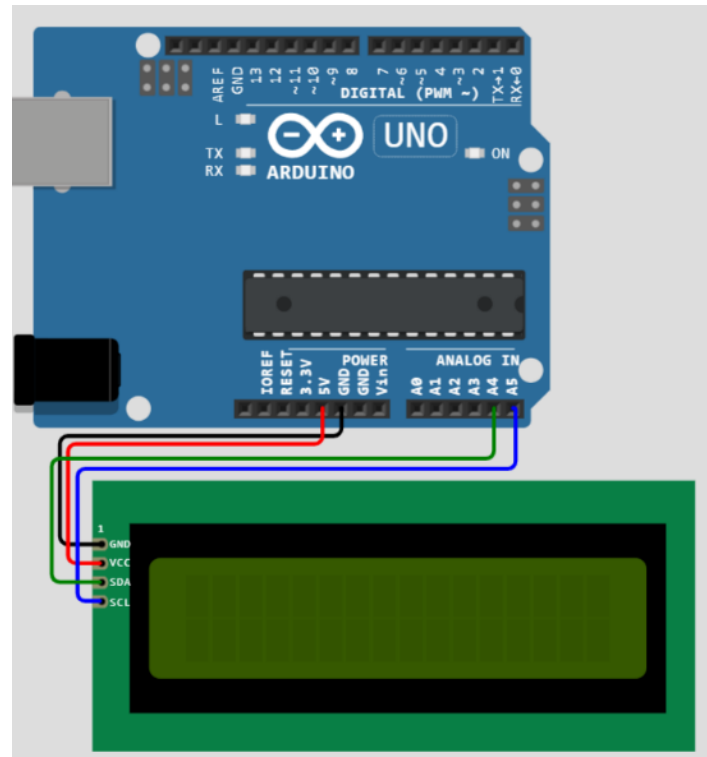


Рис. 9. Подключение LCD-дисплея через i2c.



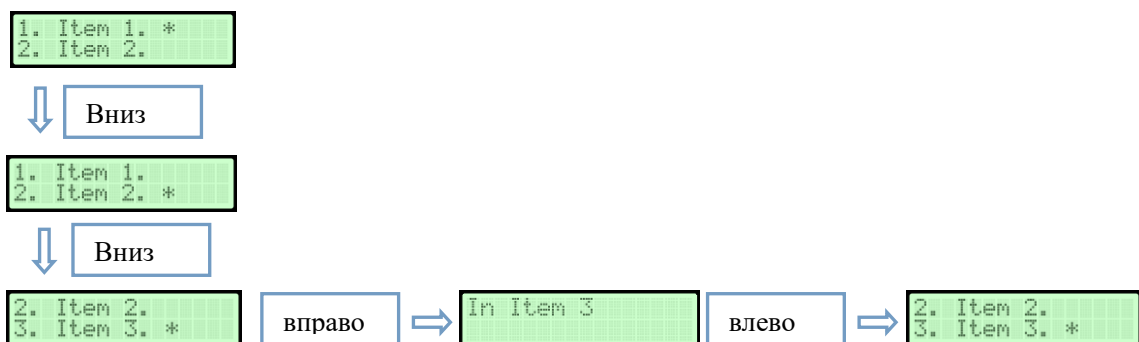
```

{
  lcd.setCursor(0, 1); // перемещаем курсор на начало второй строки
  lcd.print(millis() / 1000); // Выводим на экран кол-во секунд с момента запуска ардуины
}

```

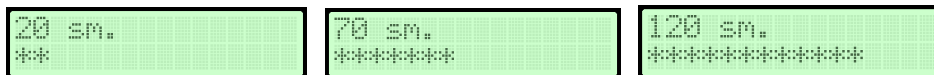
Задания

1. Схема состоит из экрана и кнопки. Сделать прототип таймера: нажатие кнопки запускает таймер, второе нажатие останавливает таймер, следующее нажатие начинает таймер с начала.
2. Схема состоит из экрана и 2-х кнопок. Сделать прототип таймера: нажатие первой кнопки запускает таймер, второе нажатие на эту же кнопку ставит таймер на паузу, следующее нажатие на эту же кнопку продолжает таймер с остановленной позиции, нажатие на вторую кнопку сбрасывает таймер в исходное состояние.
3. Схема состоит из экрана и 4 кнопок: вверх, вниз, влево, вправо. Нужно сделать прототип меню: в начальном состоянии на экране 2 пункта меню, активным (*) является первый пункт. При нажатии кнопки вниз активным становится второй пункт, потом 3-й, потом снова 1-й. При нажатии кнопки вверх активным становится предыдущий пункт, после 1-го пункта – последний. При нажатии кнопки вправо – текущий пункт меню выполняется. Возврат в меню – кнопка влево.

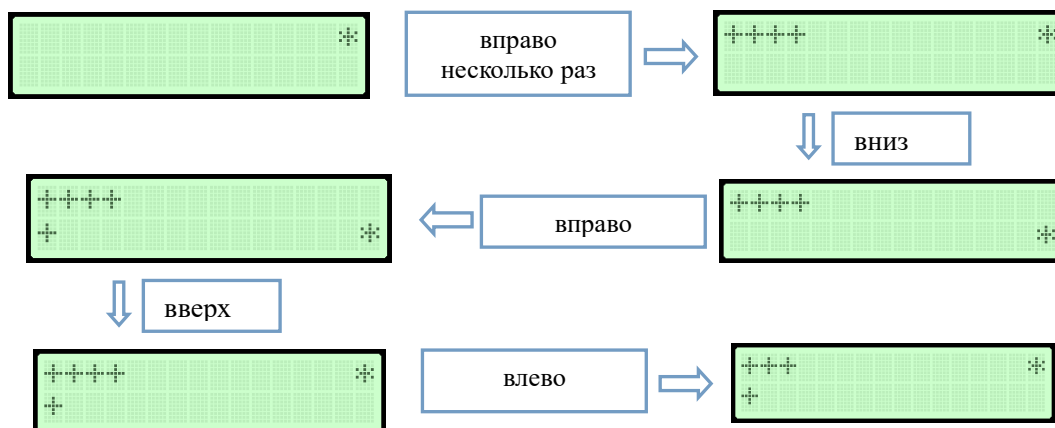


4. Схема состоит из экрана, 3-х кнопок и сервомотора. В начальном состоянии на экране число 0. По нажатию первой кнопки число на экране увеличивается на 10, по нажатию второй кнопки – уменьшается на 10. По нажатию 3-й кнопки сервомотор поворачивается на указанный на экране градус. Нужно предусмотреть ограничение угла поворота от 0 до 180.
5. Схема состоит из экрана, кнопки и датчика расстояния (ультразвукового или инфракрасного). По нажатию кнопки на первой строке экрана

появляется значение расстояния, измеряемого в текущий момент датчиком, а на второй строке – ряд из одинаковых символов, обозначающий это расстояние (см. скрины ниже). По второму нажатию на кнопку измерения останавливаются (значения остаются на экране, но не меняются), следующе нажатие – снова продолжают и т.д.



6. Схема состоит из экрана, потенциометра и двух кнопок (вверх и вниз). В начальном состоянии на экране только знак * в верхней строке в последней ячейке (см. скрины ниже). При перемещении потенциометра вправо строка постепенно заполняется знаками +, при перемещении влево знаки + по одному стираются. При нажатии кнопки вниз знак * перемещается на нижнюю строку и все действия происходят с ней. При нажатии кнопки вверх активной становится опять первая строка и т.д.



7. Схема состоит из экрана, потенциометра и двух кнопок (Enter, вниз). В начальном состоянии на экране ничего нет. При перемещении потенциометра вправо в первом символе появляются буквы английского алфавита (a, b, c и т.д.), при перемещении влево буквы меняются в обратном порядке. При нажатии кнопки Enter выбранная буква остается на своём месте, а виртуальный курсор перемещается на следующий символ, с ним действия аналогичные. При нажатии кнопки вниз активной становится следующая строка, с ней можно производить аналогичные действия. Таким образом для примера можно написать свои имя и фамилию на экране.

Лабораторная работа №4. Двухосный джойстик

Позволяет более плавно и точно отслеживать степень отклонения от нулевой точки (рис. 10). Помимо удобства по сравнению с кнопками, это позволяет реализовывать более совершенные интерфейсы.

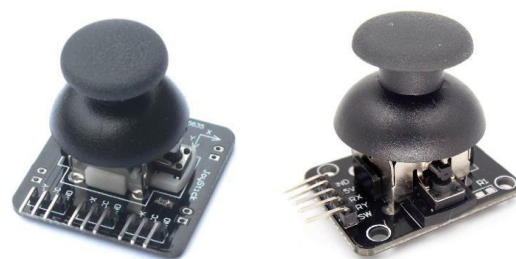


Рис. 10. Разные виды 2-х осных джойстиков.

По направлениям отклонения X (горизонтально) и Y (вертикально) на плате джойстика установлены два потенциометра. Основываясь на их показаниях, джойстик отслеживает отклонения влево, вправо, вверх, вниз и выдает аналоговый сигнал от 0 до 1023.

Ось Z посажена на кнопку и выдает цифровой сигнал (вкл/выкл).

В зависимости от производителя может быть разное количество выходов (рис. 10), но всегда есть питание, земля, выход для оси X, оси Y и простого нажатия (SW или KEY или Z), отличие в том, что может быть либо три группы проводов на каждый выход для оси X, оси Y и простого нажатия (совместно с питанием и землей), либо только 5 выходов (5 пинов).

Подключение модуля с 5 пирами предельно просто (рис. 11):

VCC – на + 5 Вольт Arduino.

GND – на любой из GND пинов Arduino.

VRx (X) и VRy (Y) или VERT и HORZ – на любые аналоговые входы Arduino.

SW или KEY или Z или Sel – к любому цифровому пину Arduino.

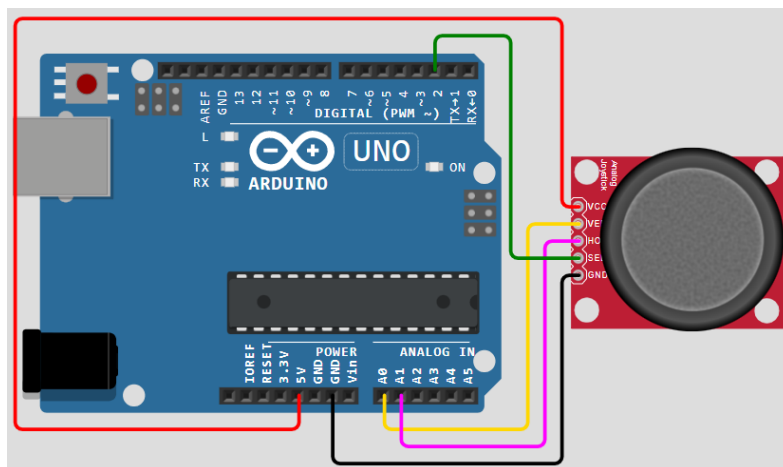


Рис. 11. Подключение 5-пинового джойстика к Arduino в Wokwi.

Для модуля с 9 выходами (3 по 3) подключение тоже простое (рис. 12):

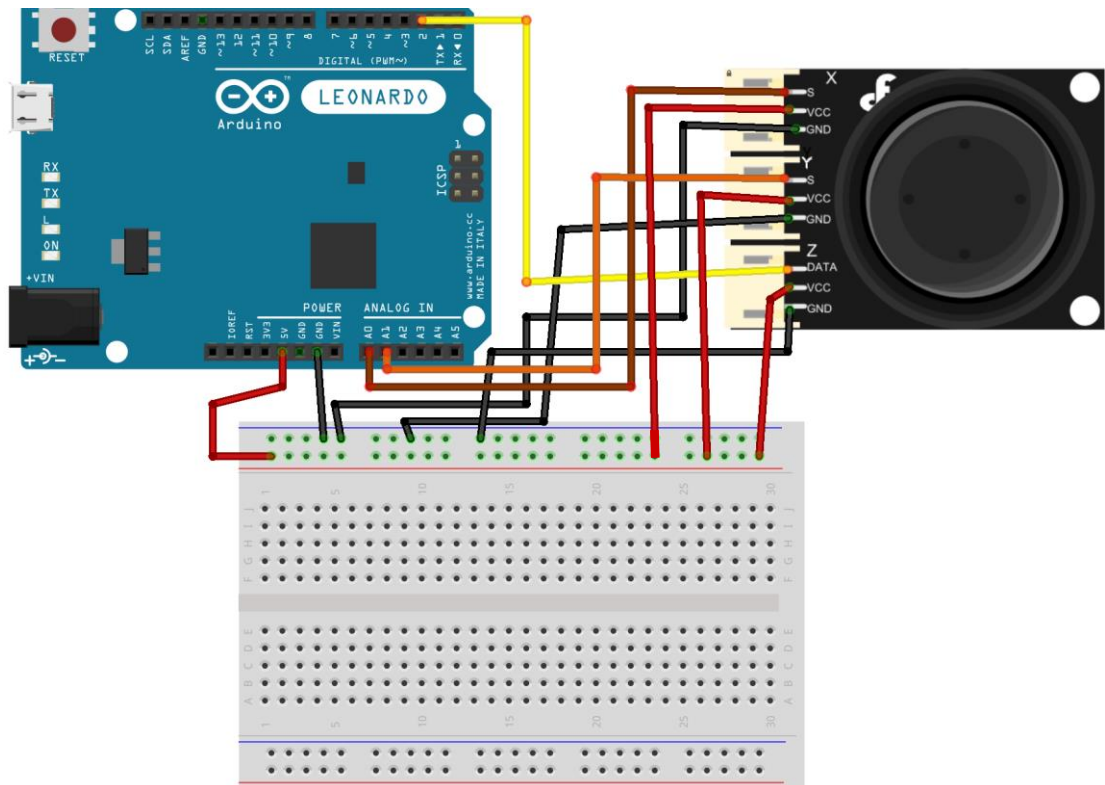


Рис. 12. Подключение модуля джойстика с 9 выходами (3 по 3) к Arduino.

Но по рис. 12 видно, что используется много проводов и макетная плата, поэтому такой модуль удобнее подключать либо через Troyka Shield, либо через Arduino Sensor Shield V5.0 (рис. 13), используя 3-х проводные шлейфы (рис. 14).



Рис. 13. Arduino Sensor Shield



Рис. 14. 3-х проводной шлейф.

Пример кода с использованием джойстика:

```
#define axis_X 0 // Ось X подключена к Analog 0
#define axis_Y 1 // Ось Y подключена к Analog 1
#define axis_Z 2 // Ось Z (кнопка) подключена к Digital 2

int value_X, value_Y, value_Z = 0; // Переменные для хранения значений осей

void setup() {
    pinMode(axis_Z, INPUT_PULLUP); // Задаем pin как вход
    Serial.begin(9600);
}

void loop() {
    value_X = analogRead(axis_X); // Считываем аналоговое значение оси X от 0 до 1023
```

```

Serial.print("X:");
Serial.print(value_X, DEC);           // Выводим значение в монитор порта

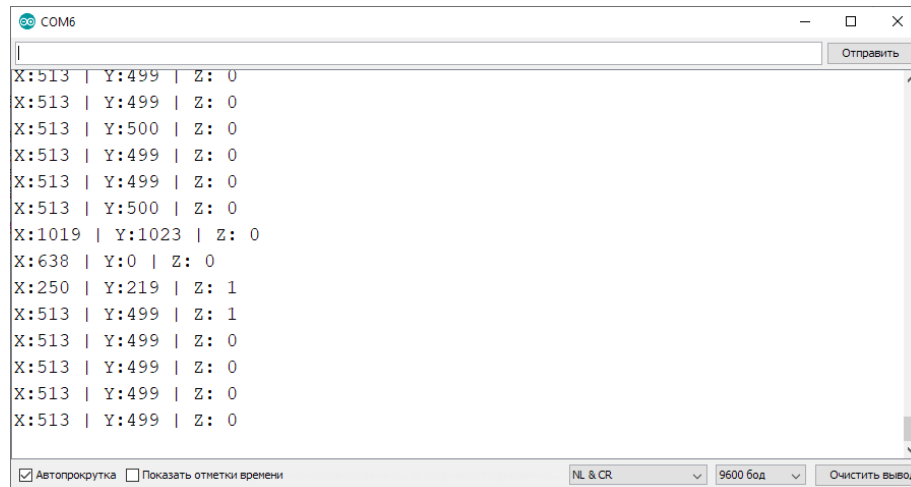
value_Y = analogRead(axis_Y);        // Считываем аналоговое значение оси Y от 0 до 1023
Serial.print(" | Y:");
Serial.print(value_Y, DEC);           // Выводим значение в монитор порта

value_Z = digitalRead(axis_Z);        // Считываем цифровое значение оси Z (кнопка) 0 или 1
value_Z = !value_Z;                  // Инвертируем значение, т.к. не нажатая кнопка – это 1
Serial.print(" | Z: ");
Serial.println(value_Z, DEC);          // Выводим значение в монитор порта

delay(250);                           // Задержка 250 мс
}

```

Пример вывода данных в монитор порта представлен на рисунке ниже.



Задания

1. Схема состоит из джойстика и 2-х сервомоторов. В исходном состоянии моторы повернуты на 90°. При перемещении джойстика по оси X меняется угол на первом серво (джойстик влево – серво поворачивает на влево на угол, соответствующий отклонению джойстика, вправо – серво поворачивает вправо), по оси Y всё то же самое происходит с другим серво.
2. Схема состоит из джойстика и 5 светодиодов. В начальном состоянии горит центральный светодиод на половину максимальной мощности. При перемещении джойстика по оси X загораются дополнительно светодиоды справа или слева, в зависимости от показаний джойстика. При перемещении по оси Y меняется яркость светодиодов (вверх – больше, вниз – меньше). Нажатие на кнопку – сброс в начальное состояние.
3. Схема состоит из джойстика и 2-х сервомоторов. При перемещении джойстика по оси X сервомотор вращается в нужную сторону (нужно

учесть ограничения сервомотора), пока джойстик отклонен от центра. По оси Y всё то же самое происходит с другим сервомотором. При нажатии на кнопку сервомоторы перемещаются на 90°.

4. Схема состоит из джойстика и светодиодной матрицы. В начальном состоянии на матрице горит первый светодиод. При перемещении джойстика точка на матрице перемещается в нужном направлении (вверх, вниз, влево, вправо), причем при выходе за край точка должна появляться с другой стороны матрицы.
5. Схема состоит из джойстика и светодиодной матрицы. В начальном состоянии на матрице горит первый ряд. При перемещении джойстика вниз – заполняется следующий ряд, при перемещении вверх – текущий ряд затухает.
6. Схема состоит из джойстика и светодиодной матрицы. В начальном состоянии на матрице горит первый столбец. При перемещении джойстика вправо – заполняется следующий столбец, при перемещении влево – текущий столбец затухает.
7. Схема состоит из джойстика и семисегментного индикатора. В начальном состоянии на индикаторе 0. При перемещении джойстика вправо цифра на индикаторе увеличивается, при перемещении влево – уменьшается. При нажатии на джойстик – сброс на 0.

Лабораторная работа №5. Работа с мини клавиатурой 4x4

Мини клавиатура 4x4 – удобный инструмент для ввода данных в Arduino. Состоит из 16 клавиш: 10 цифровых кнопок, четыре буквенные, звездочка и решетка. Такого набора с запасом хватает на большинство реальных проектов. Есть разные виды таких клавиатур (рис. 15).



Рис. 15. Мини клавиатуры 4x4.

Подключение к Arduino показано на рис. 16.

Код для определения нажатой клавиши может быть примерно следующим:

```
#include <Keypad.h>
const byte numRows = 4; //кол-во строк
const byte numCols = 4; //кол-во столбцов

//матрица, определяющая расположение эл-ов на клавиатуре
char keymap[numRows][numCols] =
{
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

//определяем подключение пинов для строк и столбцов
//в виде одномерных массивов
byte rowPins[numRows] = {9, 8, 7, 6};
byte colPins[numCols] = {5, 4, 3, 2};

//инициализируем клавиатуру
Keypad myKeypad = Keypad(makeKeymap(keymap), rowPins, colPins,
                          numRows, numCols);

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  char keypressed = myKeypad.getKey(); //получаем символ на нажатой клавише
  if (keypressed != NO_KEY) //если он не пустой
  {
    Serial.println(keypressed); //выводим его в монитор порта
  }
}
```

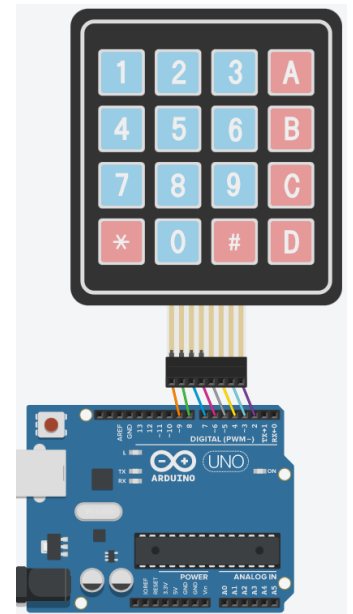


Рис. 16. Подключение мини клавиатуры к Arduino

При запуске данного скетча нажатая клавиша будет выводиться в монитор порта.

Теперь поменяем скетч, чтобы из нажатых клавиш формировалась строка и по нажатию клавиши # выводилась в монитор порта. Для этого объявим перед функцией setup массив символов и счетчик:

```
int i=0;
char data[20]; //20 - максимальная длина строки

и перепишем функцию loop() в следующем виде:

void loop()
{
  char customKey = myKeypad.getKey(); //получаем символ на нажатой клавише
  if (customKey != NO_KEY) { //если он не пустой
```

```

if(customKey == '#'){ //если это #
    Serial.println(); //то переводим курсор в мониторе порта на следующую строку
    Serial.print("data = "); //и печатаем получившуюся строку
    Serial.println(data);
    strcpy(data, ""); //а массив букв зануляем (присваиваем пустую строку)
    i=0; //и зануляем счетчик
} else { //иначе, если нажали обычную клавишу
    data[i]=customKey; //добавляем символ с клавиши в массив символов в i-ю позицию
    Serial.print(customKey); //выводим его для отладки
    i++; //и увеличиваем счетчик символов
}
}
}

```

При работе данного кода в монитор порта будет выводиться примерно следующая информация в зависимости от нажатой клавиши:

```

55558A
data = 55558A
C9874BA
data = C9874BA

```

Задания

1. Схема состоит из мини клавиатуры, двух кнопок и двустрочного LCD экрана. Сделать имитатор кодового замка: первая кнопка включает режим задания кода – в нем с мини клавиатуры вводится код, подтверждается нажатием # и запоминается в переменную (на LCD экране в первой строке написано Set, во второй – дублируется код), вторая кнопка включает режим ввода кода для проверки – с мини клавиатуры вводится код, подтверждается нажатием # и сравнивается с сохраненным в первом режиме (на LCD экране в первой строке написано Check, во второй – дублируется код и выводится результат проверки кода – true или false).
2. Схема состоит из мини клавиатуры и двустрочного LCD экрана. Сделать таймер – с помощью мини клавиатуры вводится время обратного отсчета, оно отображается на LCD экране, для запуска и паузы используется *, для сброса таймера – #.
3. Схема состоит из мини клавиатуры, двустрочного LCD экрана и сервомотора. С мини клавиатуры вводится угол (он отображается на LCD экране) и после нажатия клавиши # серво поворачивается на этот угол.
4. Схема состоит из мини клавиатуры, двустрочного LCD экрана и DC-мотора. С мини клавиатуры вводится скорость вращения – от 0 до 255 в одну сторону и те же значения со знаком * в другую сторону (все

это отображается на LCD экране) и после нажатия клавиши # мотор выполняет вращения с указанными параметрами.

5. Схема состоит из мини клавиатуры, двустрочного LCD экрана и DC-мотора. С мини клавиатуры вводится время вращения в одну сторону, после нажатия клавиши # мотор выполняет вращение с указанным временем (например, если указано 2 секунды, то он 2 секунды крутится в одну сторону, потом 2 секунды в другую сторону, и т.д.), и на экране показан отсчет времени вращения.
6. Схема состоит из мини клавиатуры, двустрочного LCD экрана и светодиодной матрицы из RGB-лент (4x4). С мини клавиатуры вводится адрес светодиода на матрице (номер строки и столбца) и цвет в формате RGB (например, 255 0 0 для красного цвета). Все вводимые величины отображаются на LCD экране (можно на разных строках). Каждый ввод подтверждается клавишей *, окончание ввода – клавишей #, после чего на матрице загорается указанный светодиод нужным цветом.
7. Схема состоит из мини клавиатуры, двустрочного LCD экрана и светодиодной матрицы из RGB-лент (4x4). С мини клавиатуры вводится число, которое означает кол-во строк, и цвет в формате RGB (например, 255 0 0 для красного цвета). Все вводимые величины отображаются на LCD экране (можно на разных строках). Каждый ввод подтверждается клавишей *, окончание ввода – клавишей #, после чего на матрице указанное кол-во строк загораются нужным цветом.

Лабораторная работа №6. Работа с графическими дисплеями

Графические дисплеи позволяют выводить практически любую информацию, в том числе и графическую, можно управлять любым пикселем.

В виртуальной среде Wokwi можно работать с двумя видами графических экранов:

1. Монохромный 128x64 OLED дисплей с i2c интерфейсом

2. Цветной 240x320 2.8" LCD-TFT дисплей с SPI-интерфейсом.

Рассмотрим работу с ними.

1. Монохромный 128x64 OLED дисплей с i2c интерфейсом

Пример подключения показан на рис.17.

Пример кода для рисования нескольких линий:

```
#include <Adafruit_GFX.h> // подключаем библиотеки
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128 // ширина
#define SCREEN_HEIGHT 64 // и высота дисплея
//создаем объект для работы с дисплеем
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT);

void setup() {
  Serial.begin(9600);
  display.begin(SSD1306_SWITCHCAPVCC, 0x3D); //инициализация дисплея (0x3D - адрес на шине i2c)
  //для экранов от фирмы Амперка адрес 0x3C
  display.clearDisplay(); //очищаем дисплей
  display.drawLine(0, 0, 127, 63, WHITE); //линия из верхнего левого угла в правый нижний
  display.display(); //отрисовка данных
  delay(2000);
  display.drawLine(127, 0, 0, 63, WHITE); //линия из верхнего правого угла в левый нижний
  display.drawLine(0, 31, 127, 31, WHITE); //линия по центру дисплея
  display.display(); //отрисовка данных
  delay(2000);
}

void loop() {
}
```

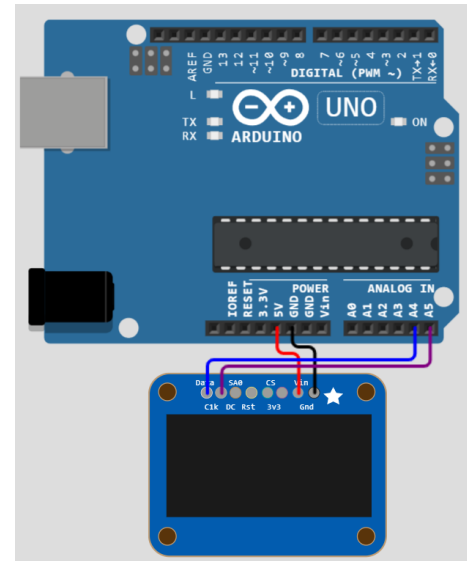
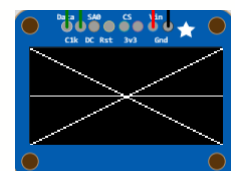


Рис. 17. 128x64 OLED дисплей с i2c интерфейсом



Для работы данного примера необходимо к проекту подключить библиотеку Adafruit_SSD1306. Для этого выбираем Library Manager, в нем жмем на кнопку +, в появившемся окне печатаем Adafruit SSD1306 и выбираем появившуюся библиотеку (рис. 18).

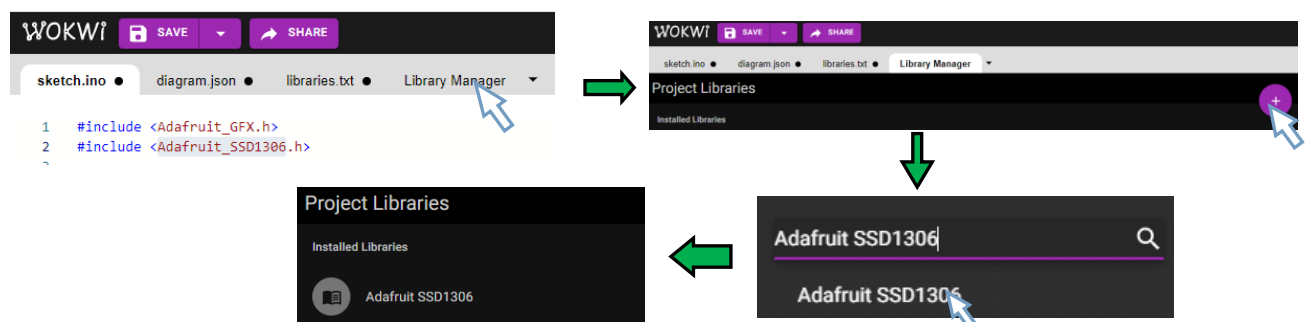


Рис. 18. Подключение библиотеки в проект на платформе Wokwi

Также нужно добавить i2c-адрес для дисплея: переходим на вкладку diagram.json, в разделе parts находим wokwi-ssd1306 и в его свойстве "attrs"

внутри фигурных скобок добавляем "i2c-address": "0x3D" (рис. 19).

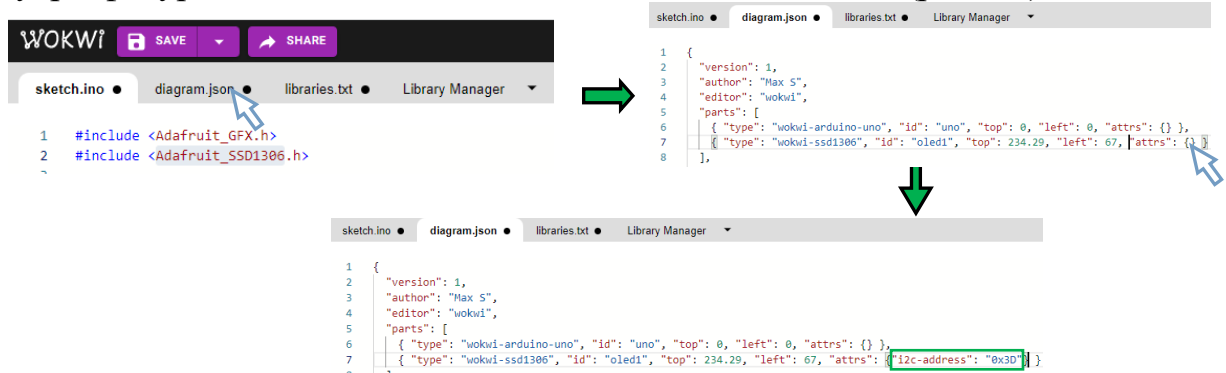


Рис. 19. Установка i2c-адреса

Некоторые графические функции для работы с дисплеем:

`drawPixel(int x, int y, uint color)` – точка нужного цвета
`drawCircle(int x0, int y0, int r, int color)` – контур окружности нужного цвета
`drawLine(int x0, int y0, int x1, int y1, int color)` – линия
`drawRect(int x, int y, int w, int h, int color)` – контур прямоугольника, верхний левый угол в точке (x, y), шириной w и высотой h
`drawTriangle(int x0, int y0, int x1, int y1, int x2, int y2, int color)` – контур треугольника по точкам
`drawRoundRect(int x, int y, int w, int h, int r, int color)` – контур скругленного прямоугольника, верхний левый угол в точке (x, y), шириной w, высотой h, радиус скругления углов r
`fillRect(int x, int y, int w, int h, int color)` – заполненный прямоугольник
`fillCircle(int x0, int y0, int r, int color)` – заполненный круг
`fillTriangle(int x0, int y0, int x1, int y1, int x2, int y2, int color)` – заполненный треугольник
`fillRoundRect(int x0, int y0, int w, int h, int radius, int color)` – заполненный скругленный прямоугольник
`drawBitmap(int x, int y, const uint8_t bitmap[], int w, int h, int color)` – вывод изображения

Пример с движением объекта:

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128 // ширина
#define SCREEN_HEIGHT 64 // и высота дисплея
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT);
long time = 0; //переменная для отчета времени
byte i = 0; //переменная для перемещения окружности

void setup() {
  Serial.begin(9600);
  display.begin(SSD1306_SWITCHCAPVCC, 0x3D); //инициализация дисплея (иногда 0x3C)
  display.clearDisplay();
  time = millis(); //фиксируем время в начале программы
}

void loop() {
  display.drawCircle(11 + i, 11, 10, WHITE); //рисует окружность
  display.display();
  if (millis() - time >= 50) { //если с прошлого замера времени прошло 50 миллисекунд
    time = millis(); //делаем новый замер времени
    display.drawCircle(11 + i, 11, 10, BLACK); //рисует окружность черным цветом (цветом фона)
    display.display();
    i++; //увеличиваем переменную для перемещения
    if (i + 11 >= 140) i = 0; //если выйдем за пределы экрана, то возвращаемся в начало
```

```

}
}

```

В результате выполнения этого кода на экране будет двигаться окружность от левого края к правому (рис. 20).



Рис. 20. Движение окружности на дисплее.

2. Цветной 240x320 2.8" LCD-TFT дисплей с SPI-интерфейсом.

Пример подключения показан на рис.21. Соответствие пинов дисплея и Arduino приведено в таблице:

№	Обозначение на дисплее	Пин Arduino
1	VCC	5V
2	GND	GND
3	CS	10
4	RST	-
5	D/C	9
6	MOSI	11
7	SCK	13
8	LED	
9	MISO	12

Пример с рисованием линии, прямоугольника и окружности (рис. 22):

```

#include "SPI.h"
#include "Adafruit_GFX.h"
#include "Adafruit_ILI9341.h"
//обозначение пинов (эти два могут отличаться)
#define TFT_DC 9
#define TFT_CS 10
//остальные пины всегда должны быть как в таблице
//создаем объект дисплея
Adafruit_ILI9341 tft = Adafruit_ILI9341(TFT_CS, TFT_DC);

void setup() {
  Serial.begin(9600);
  Serial.println("ILI9341 Test!");
  tft.begin(); //начинаем работать с дисплеем
  //заполняем дисплей белым цветом
  tft.fillScreen(ILI9341_WHITE);
  //красная линия из верхнего левого угла в правый нижний
  tft.drawLine(0, 0, tft.width(), tft.height(), ILI9341_RED);
  //синяя окружность по центру дисплея
  tft.drawCircle( tft.width()/2, tft.height()/2, 60, ILI9341_BLUE);
  //черный прямоугольник по центру дисплея
  tft.drawRect(tft.width()/2-70, tft.height()/2-90, 140, 180,
  ILI9341_BLACK);
  delay(2000);
}

```

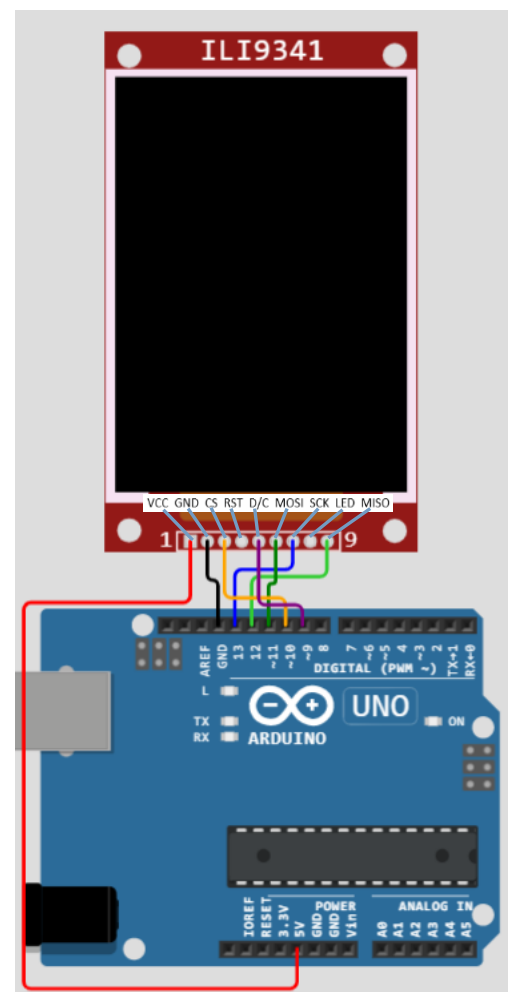


Рис. 21. Подключение LCD-TFT дисплея с SPI-интерфейсом

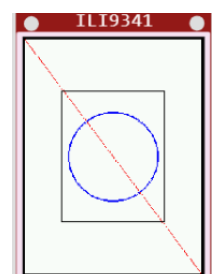


Рис. 22. Пример рисования.

```
void loop(void) {
}
```

Все основные функции для рисования такие же, как у предыдущего [дисплея](#), но теперь можно указывать разные цвета:

Константа	Аналог в формате RGB
ILI9341_BLACK	0, 0, 0
ILI9341_NAVY	0, 0, 123
ILI9341_DARKGREEN	0, 125, 0
ILI9341_DARKCYAN	0, 125, 123
ILI9341_MAROON	123, 0, 0
ILI9341_PURPLE	123, 0, 123
ILI9341_OLIVE	123, 125, 0
ILI9341_LIGHTGREY	198, 195, 198
ILI9341_DARKGREY	123, 125, 123
ILI9341_BLUE	0, 0, 255
ILI9341_GREEN	0, 255, 0
ILI9341_CYAN	0, 255, 255
ILI9341_RED	255, 0, 0
ILI9341_MAGENTA	255, 0, 255
ILI9341_YELLOW	255, 255, 0
ILI9341_WHITE	255, 255, 255
ILI9341_ORANGE	255, 165, 0
ILI9341_GREENYELLOW	173, 255, 41
ILI9341_PINK	255, 130, 198

Задания

(с любыми видами графических экранов):

Часть I (статика)

1. Нарисуйте три окружности с заключёнными в них треугольником, квадратом и звездой.
2. Нарисуйте контур прямоугольника, в котором нарисуйте линиями своё имя или фамилию.
3. Нарисуйте на экране 4 ромба.
4. Нарисуйте на экране 4 параллелограмма.
5. Нарисуйте на экране 4 треугольника.
6. Нарисуйте на экране сетку из линий.
7. Нарисуйте на экране окружность, разделенную на 4 сектора.

Часть II (динамика)

Схема состоит из джойстика и дисплея.

1. В начальном состоянии на дисплее по центру нарисована окружность. При перемещении джойстика окружность перемещается в нужном направлении и возвращается обратно, если джойстик возвращается в исходное состояние.
2. В начальном состоянии на дисплее по центру нарисован прямоугольник. При перемещении джойстика прямоугольник перемещается в нужном направлении и остается там, если джойстик возвращается в исходное состояние.
3. В начальном состоянии на дисплее по центру нарисована окружность. При перемещении джойстика окружность начинает перемещаться в нужном направлении и не останавливается, только меняет направление

перемещения в зависимости от положения джойстика.

4. Нужно сделать простейший графопостроитель: при перемещении джойстика на дисплее рисуется линия в нужном направлении.
5. В начальном состоянии на дисплее по центру нарисован квадрат. При перемещении джойстика влево или вправо квадрат соответственно увеличивается или уменьшается, при перемещении джойстика вверх или вниз квадрат передвигается вверх или вниз.
6. В начальном состоянии на дисплее по центру нарисован прямоугольник. При перемещении джойстика прямоугольник меняет свои размеры в соответствии с направлением джойстика: вверх – увеличивается вверх, вниз – увеличивается вниз и т.д.
7. В начальном состоянии на дисплее в верхнем левом углу нарисована окружность. При нажатии джойстика вниз она начинает двигаться вниз, если дойдет до края экрана – останавливается и ждет дальнейшего движения джойстика. Соответственно, при нажатии джойстика вверх окружность идет до верхнего края, влево – до левого края, вправо – до правого края. Нажатие кнопки джойстика сбрасывает систему в начальное положение.

Лабораторная работа №7. ИК-модуль и пульт ДУ

ИК-модуль (рис. 23) позволяет принимать сигналы в ИК-диапазоне. Теоретически, можно использовать с любым пультом ДУ, но есть специальные пульты, идущие как правило в комплекте с ИК-модулем (рис. 24).

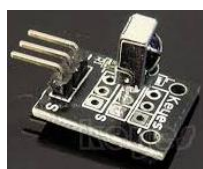


Рис. 23. Разные виды ИК-модулей.



Рис. 24. Разные виды пультов.

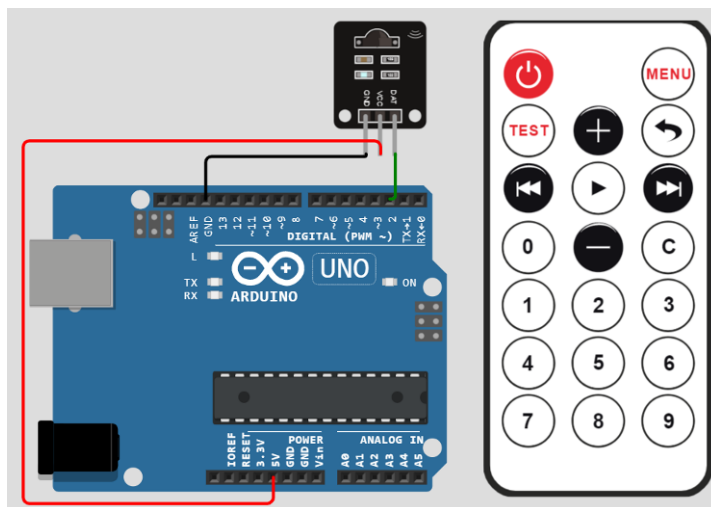


Рис. 25. Схема проекта с ИК-модулем и пультом.

Пульты отличаются кодом передаваемых сигналов. Рассмотрим пример на платформе Wokwi. Схема проекта показана на рис. 25. Код проекта:

```
#include <IRremote.h> // библиотека для ИК-модуля
```

```

int RECV_PIN = 2; //пин для ИК-модуля

void setup()
{
  Serial.begin(9600);
  IrReceiver.begin(RECV_PIN); // запускаем прием данных по ИК-модулю
  Serial.println("IR example");
}

void loop() {
  if (IrReceiver.decode()) { // если поступил сигнал на ИК-модуль
    int IRcode = IrReceiver.decodedIRData.command; //получаем его код
    Serial.print("code = "); //выводим код
    Serial.println(IRcode);
    if (IRcode == 48) { //48 - это кнопка с цифрой «1»
      Serial.print("    key = ");
      Serial.println("1");
    }
    if (IRcode == 104 ) { //104 - это кнопка с цифрой «0»
      Serial.print("    key = ");
      Serial.println("0");
    }
  }
  IrReceiver.resume(); // возобновляем прием данных
}
delay(100);
}

```

В итоге в монитор порта будет примерно следующий вывод:

```

IR example
code = 104
    key = 0
code = 48
    key = 1
code = 74
code = 56

```

Таблица соответствия кнопок на пульте и кодов (на платформе Wokwi):

Кнопка на пульте	Код	Кнопка на пульте	Код
Power	162	C	176
Menu	226	1	48
Test	34	2	24
Plus	2	3	122
Back	194	4	16
Previous	224	5	56
Play	168	6	90
Next	144	7	66
0	104	8	74
Minus	152	9	82

По примеру видно, что для программирования реакции на нужную клавишу нужно прописывать отдельный блок условия. Но если нам необходимо отобразить нужное действие на экране, то можно воспользоваться библиотекой [Dictionary](#) и сделать небольшой словарь значений (аналог HashMap в Java).

Сделаем схему проекта как на рис.26.

Код примера:

```
#include <IRremote.hpp> // библиотека для ИК-модуля
#include <LiquidCrystal_I2C.h> // библиотека для LCD-экрана с i2c
#include <Dictionary.h> // библиотека для словаря
#define PIN_RECEIVER 2 // пин для ИК-модуля
// IrRecv receiver(PIN_RECEIVER); // объект для ИК-модуля
LiquidCrystal_I2C lcd(0x27, 16, 2); // объект для LCD-экрана с i2c
Dictionary *dict = new Dictionary(20); // словарь для 20 значений
char str[6] = ""; // массив символов для кода кнопки
void setup()
{
    lcd.init();
    lcd.backlight();
    Serial.begin(9600);
    Serial.println("hello");
    lcd.print("<press a button>"); // начальная надпись на экране
    dict->insert("162", "power"); // вставляем в словарь коды кнопок и их значение
    dict->insert("226", "Menu");
    dict->insert("34", "Test");
    dict->insert("2", "Plus");
    dict->insert("194", "Back");
    dict->insert("224", "Previous");
    dict->insert("168", "Play");
    dict->insert("144", "Next");
    dict->insert("104", "0");
    dict->insert("152", "Minus");
    dict->insert("176", "C");
    dict->insert("48", "1");
    dict->insert("24", "2");
    dict->insert("122", "3");
    dict->insert("16", "4");
    dict->insert("56", "5");
    dict->insert("90", "6");
    dict->insert("66", "7");
    dict->insert("74", "8");
    dict->insert("82", "9");
    // receiver.enableIRIn(); //запускаем прием данных по ИК-модулю
    IrReceiver.begin(PIN_RECEIVER);
}
```

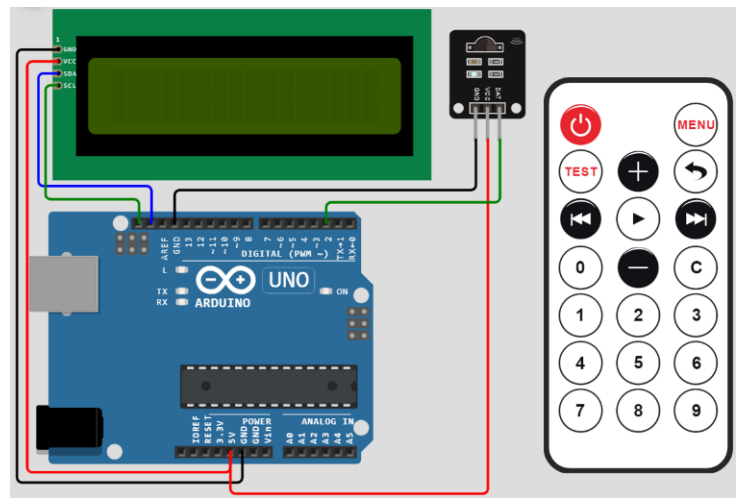


Рис. 26. ИК-модуль, пульт и LCD-экран с i2c

```

void loop()
{
    if (IrReceiver.decode()) { // если поступил сигнал на ИК-модуль
        translateIR(); //вызываем функцию по обработке сигнала
        IrReceiver.resume(); // возобновляем прием данных
    }
}

void translateIR() // функция по обработке сигнала
{
    int IRcode = IrReceiver.decodedIRData.command; //записываем код сигнала в переменную
    Serial.println(IRcode);
    //получаем из словаря значение кнопки (тип String) и переводим его в массив символов (char*)
    dict->search(String(IRcode)).toCharArray(str, 6);
    lcdPrint(str); //передаем массив символов со значением кнопки в функцию для вывода
} //на экран

void lcdPrint(char* text) // функция для вывода значения и кода кнопок на LCD-экран
{
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("button pressed:");
    lcd.setCursor(0, 1);
    lcd.print(text); // значение кнопки
    lcd.print(" code: ");
    lcd.print(IrReceiver.decodedIRData.command); // код кнопки
}

```

Пример работы показан на рис. 27.

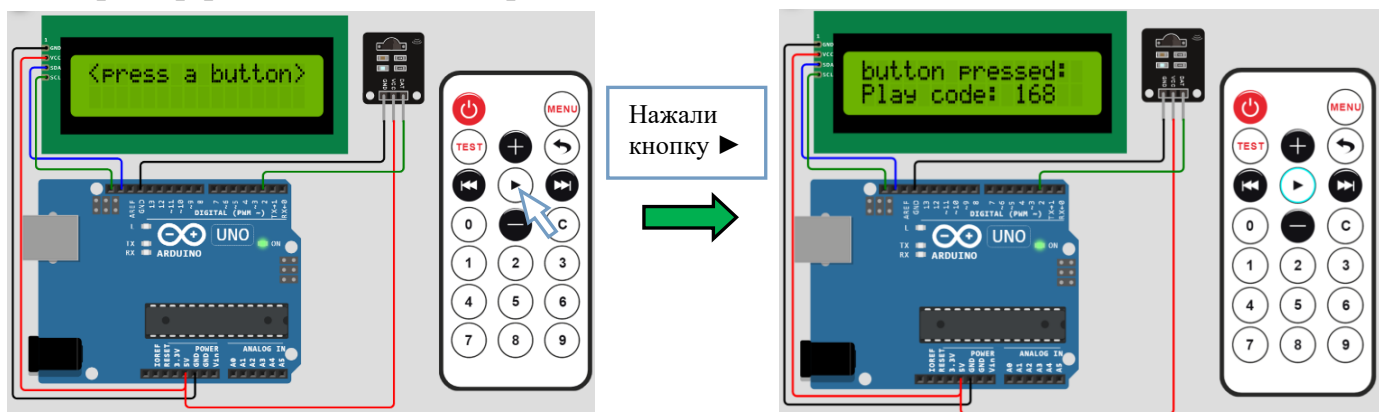


Рис. 27. Показ нажатой кнопки на LCD-экране

Задания

1. Схема состоит из ИК-модуля, пульта и сервомотора. Запрограммировать возможность управлять сервомотором с помощью кнопок пульта, чтобы вращать мотор влево, вправо и сбрасывать на начальную, конечную и центральную позицию.
2. Схема состоит из ИК-модуля, пульта и семисегментного индикатора. Запрограммировать возможность вывода на индикатор нажатой на пульте цифры, запускать отсчет от 0 до 9 и обратно.
3. Схема состоит из ИК-модуля, пульта и RGB-светодиода. Запрограммировать возможность управлять светодиодом: зажечь нужный цвет (минимум 3), запустить изменение цвета (красный, зеленый, синий), мигание (1 раз красным, 2 раза зеленым, 3 раза синим).

4. Схема состоит из ИК-модуля, пульта и 8 светодиодов, подключенных через сдвиговый регистр. Запрограммировать возможность управлять этими светодиодами: вкл/выкл нужного светодиода, запуск анимации (бегущий огонь, поочередное зажигание и затухание каждого светодиода, вкл/выкл светодиодов парами).
5. Схема состоит из ИК-модуля, пульта и графического дисплея из лаб.раб.№6. Запрограммировать возможность управления изображением на экране: в зависимости от нажатой кнопки на пульте рисование на экране квадрата, круга, треугольника.
6. Схема состоит из ИК-модуля, пульта и двустрочного LCD-дисплея. Запрограммировать эмуляцию управления телевизором: при нажатии на пульте кнопок + и – на дисплее должно отображаться слово Volume и значение громкости от 0 до 100 (значение должно сохраняться между вызовами), при нажатии на цифровые кнопки должно отображаться слово Program и номер нажатой цифры, кнопки  и  должны соответственно увеличивать и уменьшать номер канала.
7. Схема состоит из ИК-модуля, пульта и графического дисплея из лаб.раб. №6. Запрограммировать возможность управления движением объекта на экране: в зависимости от нажатой кнопки на пульте объект двигается вверх, вниз, вправо, влево, отдельная кнопка должна центровать объект на экране.

Лабораторная работа №8. Передача данных с компьютера на устройство и обратно

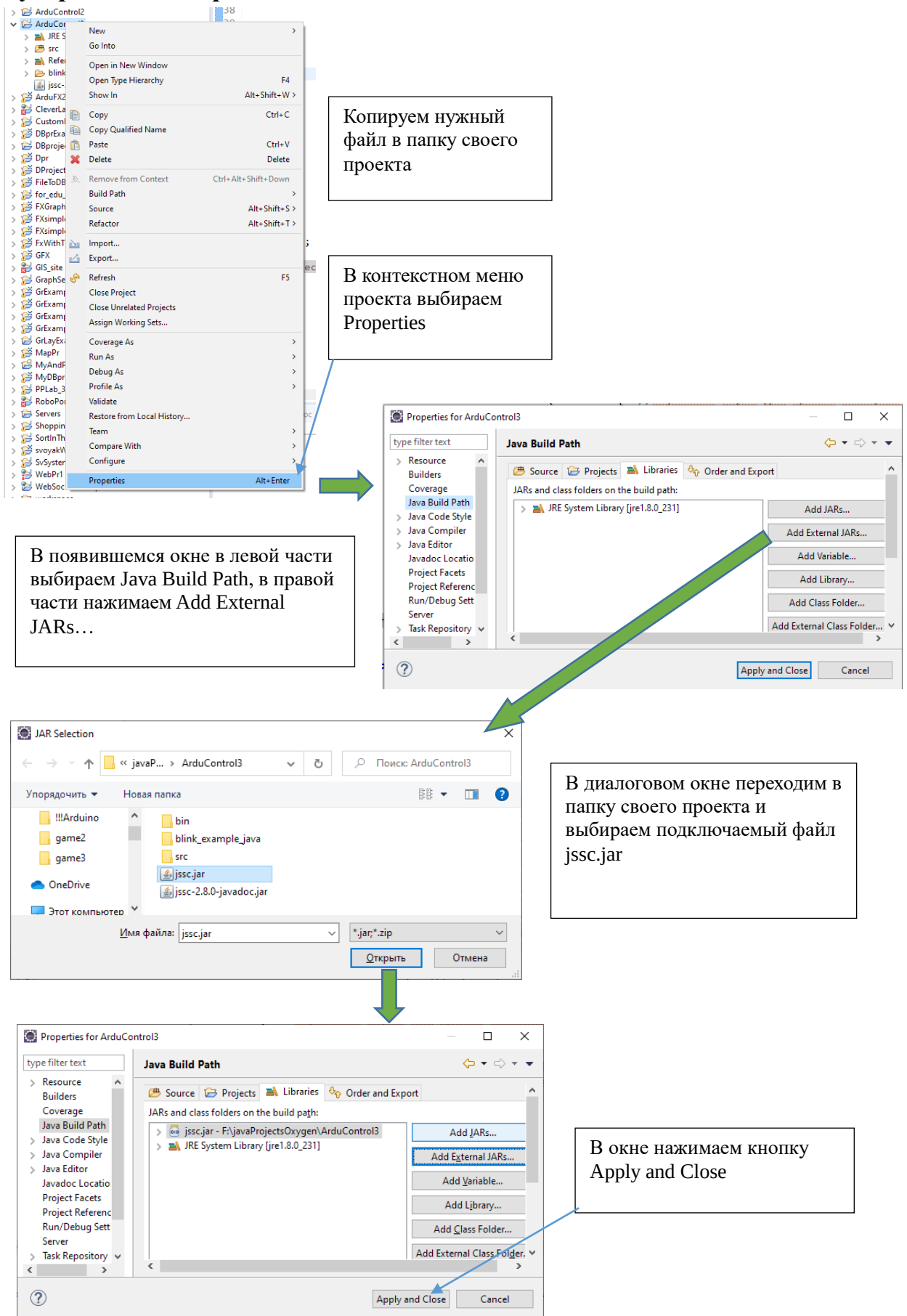


Рис. 28. Подключение библиотеки к проекту.

Рассмотрим возможности передачи данных из программы на языке Java,

запущенной на компьютере, в плату Arduino, подключенную через USB к этому компьютеру.

Для этого можно использовать библиотеку jSSC (<https://github.com/java-native/jssc/releases>). Можно взять на сервере файл jssc-2.9.2.jar и добавить к проекту в Eclipse (рис. 28). Там же лежит файл jssc-2.9.2-javadoc.jar – это документация к библиотеке. Его можно подключить к выбранному файлу (рис. 29), чтобы видеть подсказки по методам библиотеки в IDE.

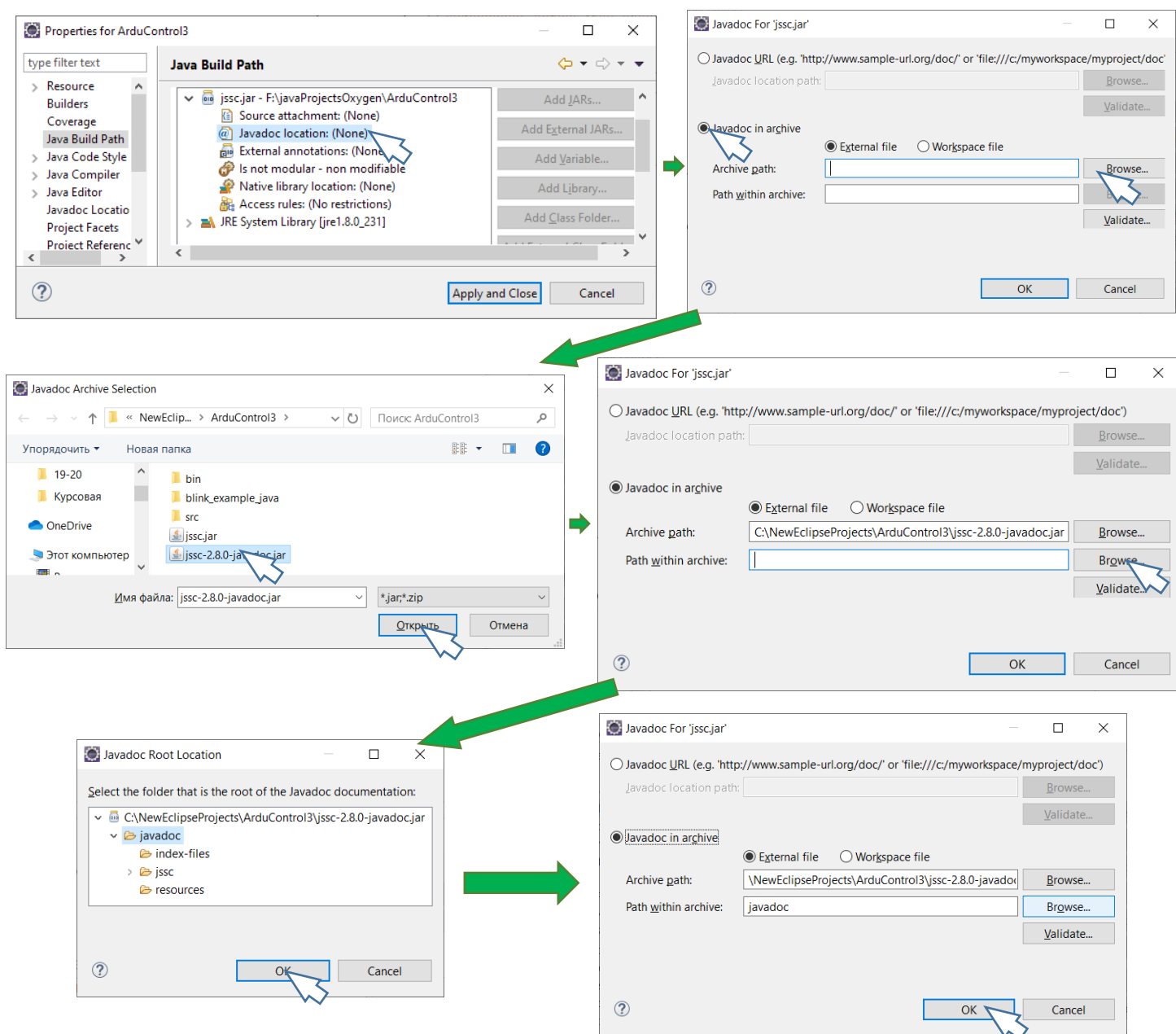


Рис. 29. Подключение документации к библиотеке.

Напишем простую программу, в которой будет выбор порта, к которому подключена плата Arduino, и чек-бокс, нажатие на который включает или выключает светодиод на плате (который подключен к пину 13). В свою очередь,

к плате Arduino также подключена кнопка, нажатие на которую включает или выключает светодиод и в зависимости от этого в интерфейсе программы ставится или убирается галочка в чек-боксе (рис. 30):

```
import java.awt.BorderLayout;
import javax.swing.JCheckBox;
import javax.swing.JComboBox;
import javax.swing.JFrame;
```

```
import jssc.SerialPort;
import jssc.SerialPortException;
import jssc.SerialPortList;
```

```
public class LedControl {
    static SerialPort serialPort = null;

    public static void main(String[] args) {
        init();
    }

    private static void init() {
        JFrame myFrame = new JFrame("ArduinoControl");
        myFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JCheckBox check = new JCheckBox("LED on/off"); // чекбокс, который будет отвечать за
                                                    //вкл/выкл светодиода на плате Arduino
        check.setEnabled(false); //делаем неактивным чекбокс, пока не выберем порт для связи с
                                //Arduino
        String[] portNames = SerialPortList.getPortNames(); // получаем список портов
        JComboBox<String> comPorts = new JComboBox<>(portNames); //создаем комбобокс с этим
                                                                //списком
        comPorts.setSelectedIndex(-1); //чтоб не было выбрано ничего в комбобоксе

        comPorts.addActionListener(arg -> { //слушатель выбора порта в комбобоксе
            // получаем название выбранного порта
            String chosenPort = comPorts.getItemAt(comPorts.getSelectedIndex());
            //если serialPort еще не связана с портом или текущий порт не равен выбранному в комбо-боксе
            if (serialPort == null || !serialPort.getPortName().contains(chosenPort)) {
                serialPort = new SerialPort(chosenPort); //задаем выбранный порт
                check.setEnabled(true); //активируем чек-бокс
                try { //тут секция с try...catch для работы с портом
                    serialPort.openPort(); //открываем порт
                    //задаем параметры порта, 9600 - скорость, такую же нужно задать для Serial.begin в Arduino
                    serialPort.setParams(9600, 8, 1, 0); //остальные параметры стандартные
                    //слушатель порта для приема сообщений от ардуино
                    serialPort.addEventListener(event -> {
                        if (event.isRXCHAR()) { // если есть данные для приема
                            try { //тут секция с try...catch для работы с портом
                                //считываем данные из порта в строку
                                String str = serialPort.readString();
                                //убираем лишние символы (типа пробелов, которые могут быть в принятой строке)
                                str = str.trim();
                                //проверяем принятую строку, и либо ставим, либо убираем галочку в чек-боксе
                                if (str.contains("1")) check.setSelected(true);
                                if (str.contains("0")) check.setSelected(false);
                                System.out.println(str); //выводим принятую строку
                            } catch (SerialPortException ex) { //для обработки возможных ошибок
                                System.out.println(ex);
                            }
                        }
                    });
                } catch (SerialPortException e) { //для обработки возможных ошибок
                    e.printStackTrace();
                }
            } else {
                System.out.println("Same port!!"); //если выбрали в списке тот же порт, что и
                                                    //до этого
            }
        });

        check.addActionListener(arg0 -> { //обработка нажатия на чек-бокс
            try { //тут секция с try...catch для работы с портом
```

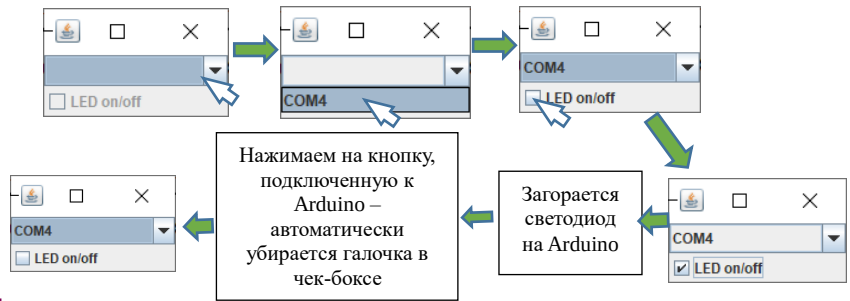


Рис. 30. Работа программы.

```

        if (check.isSelected()) { //если стоит галочка
            System.out.println("LED on"); //то выводим текст
            serialPort.writeString("1");// и отправляем символ "1" в порт
        } else {
            System.out.println("LED off");
            serialPort.writeString("0");// иначе отправляем символ "0" в порт
        }
    } catch (SerialPortException e) { //для обработки возможных ошибок
        e.printStackTrace();
    }
});
myFrame.add(comPorts, BorderLayout.NORTH); //добавляем всё на форму
myFrame.add(check, BorderLayout.CENTER);
myFrame.setLocationRelativeTo(null);
myFrame.pack();
myFrame.setVisible(true);
}
}

```

Скетч для Arduino:

```

char recv = '0'; //переменная для приема и отправки сообщений
char lastRecv = '0'; //дополнительная переменная для хранения предыдущего значения recv
byte buttonPress = HIGH; // переменная для определения нажата кнопка или нет, HIGH – не нажата
long time; // переменная для таймера
void setup() {
    pinMode(13, OUTPUT); //настраиваем пин для встроенного светодиода
    pinMode(8, INPUT_PULLUP); //настраиваем пин для кнопки
    digitalWrite(13, LOW); //гасим светодиод на всякий случай
    Serial.begin(9600); //задаем скорость порта
    time = millis(); //стартуем таймер
}

void loop() {
    if (Serial.available() > 0) { //если есть данные для приема из ком-порта
        recv = Serial.read(); //считываем 1 символ в переменную recv
    } //можно использовать Serial.readString() для чтения строк
    buttonPress = digitalRead(8); //считываем значение пина с кнопкой
    //если кнопка нажата и с прошлого нажатия прошло не менее 500 мс
    if (buttonPress == LOW && millis() - time > 500) {
        recv = (recv == '0') ? '1' : '0'; //то меняем значение переменной recv
        time = millis(); //и сбрасываем таймер
    }

    if (recv == '1') { //в зависимости от значения переменной recv
        digitalWrite(13, HIGH); //либо включаем светодиод
    }
    else {
        digitalWrite(13, LOW); //либо выключаем светодиод
    } //это можно сделать и одной строчкой
    // (recv == '1') ? digitalWrite(13, HIGH) : digitalWrite(13, LOW);

    if (lastRecv != recv) { //и если предыдущее значение recv отличается от текущего
        Serial.print("recv="); //то отправляем в ком-порт строку со значением переменной
        Serial.println(recv);
        lastRecv = recv; //и делаем одинаковыми lastRecv и recv
    } //такая конструкция нужна, чтобы отправить данные в порт 1 раз, а не постоянно
}

```

Задание

Сделать десктоп-программу (на Java или другом языке, но на Java будет оценена выше 😊) для управления любым из ранее созданных проектов из предыдущих лабораторных работ. Обязательно должна быть и отправка данных на Arduino, и получение значений от Arduino.